Ayan Salmanova

Build a GPT from Scratch — Homework Report

This project implements a byte-level Byte Pair Encoding (BPE) tokenizer and a decoder-only GPT language model trained on the Tiny Shakespeare dataset. The work includes tokenizer construction, Transformer implementation, model training, a bigram baseline, KV-cache generation, model-scaling experiments, and controlled architectural and training ablations. The report summarizes the implementation, experimental results, decoding performance, and reflections on the main findings.

1. Implementation

Tokenizer (Part A). I implemented byte-level BPE using the 256 possible byte values as base tokens. Training on the training split only, the tokenizer learned 768 merges to create a vocabulary of 1,024 tokens. Encoding applies merges in learned order, and decoding reconstructs the UTF-8 byte sequence.

Model (Part B). I implemented a decoder-only Transformer with learned token and positional embeddings, causal multi-head self-attention, Pre-LayerNorm blocks, an MLP with GELU, residual connections, dropout, and tied token-embedding/output weights. The reference M model uses 4 layers, 4 heads, embedding dimension 128, context length 128, and dropout 0.1, with 936,192 parameters.

Training (Part C). The dataset was split by character count into 90% training text (1,003,854 characters) and 10% holdout shard (111,540 characters). Training used AdamW, gradient clipping, warmup followed by cosine learning-rate decay, periodic evaluation on 50 batches, and best-checkpoint selection. Each GPT run was configured for 3,000 steps.

Inference (Part D). I implemented autoregressive generation with temperature, top-k/top-p sampling, and KV caching. Greedy outputs matched with and without caching. On the measured CPU benchmark, cached decoding reached 639.59 tokens/s versus 336.45 tokens/s uncached, a 1.90× speedup.

2. Experimental Results

Run	Parameters	Best holdout-shard loss
Bigram	1,048,576*	4.9260
S	172,672	4.2807
M	936,192	3.7875
L	2,880,384	3.6799
A1	919,808	3.9688
A2	936,192	3.8188
A3	936,192	4.2521
A4	936,192	3.7668
A5	936,192	3.9536

**Bigram parameter count is copied from the recorded project summary; it is not directly comparable to GPT parameter counts.*

The scaling runs improved from S to M to L, with L achieving the lowest recorded holdout-shard loss. A3, which uses a higher maximum learning rate, performed worse than M. A4 with dropout 0.0 slightly outperformed M, while A5 with dropout 0.2 performed worse.

3. Reflections

R1 — Most time-consuming bug

The most time-consuming bug involved handling the ablation settings for removing positional embeddings and using Post-LayerNorm. The symptom was unexpected convergence behavior in experiments A1 and A2. I investigated the experiment logs and updated the model implementation in commit 05bb94b.

R2 — Most surprising result

The dropout ablation produced a surprising result. A4 (dropout 0.0) achieved a best holdout-shard loss of 3.7668, slightly better than M at 3.7875. A5 (dropout 0.2) reached 3.9536. These results suggest that increasing dropout did not help in this particular setup.

R3 — Largest change in holdout loss

Comparing the bigram baseline (loss 4.9260) with run L (loss 3.6799) gives a reduction of 1.2461. This comparison shows the benefit observed when moving from the count-based baseline to the GPT model. Among the GPT ablations, A3 reached 4.2521 compared with M at 3.7875, a difference of 0.4645.

R4 — Generated samples and failure pattern

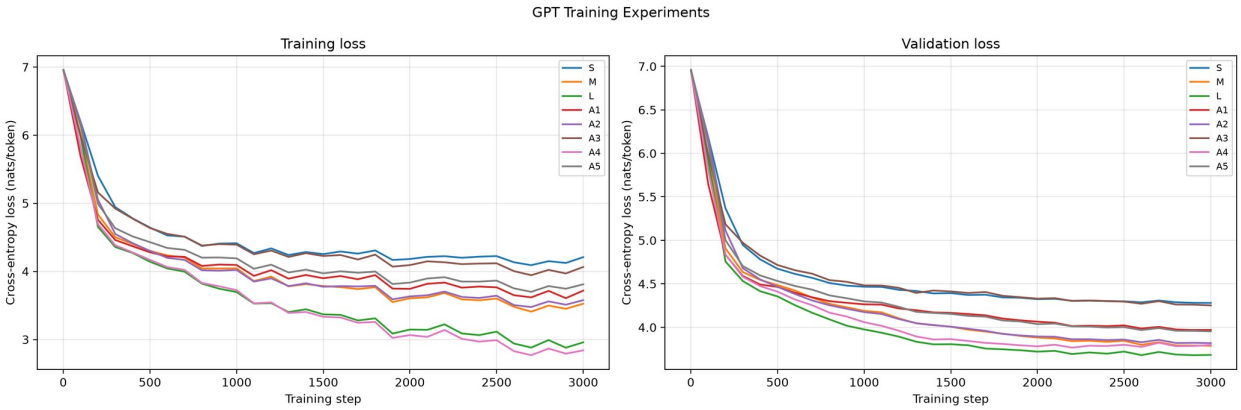
The generated samples imitate Shakespearean dialogue formatting, including character names followed by colons, but also contain nonsensical words such as “Northumber-settle.” This suggests the model learns local stylistic patterns without consistently maintaining coherent meaning. The limited context length of 128 tokens and relatively small model capacity may contribute to this behavior.

R5 — Tokenizer merges and bytes per token

The first ten merges are “e ”, “th”, “t ”, “s ”, “d ”, “, ”, “ou”, “er”, “in”, and “y ”. These frequent character pairs and short sequences allow the tokenizer to represent common patterns with fewer tokens. The recorded holdout-shard bytes-per-token value is 2.39. A separate measurement over the full input file gave 2.5083 bytes per token.

4. Learning Curves, Model Scaling, and Generation Performance

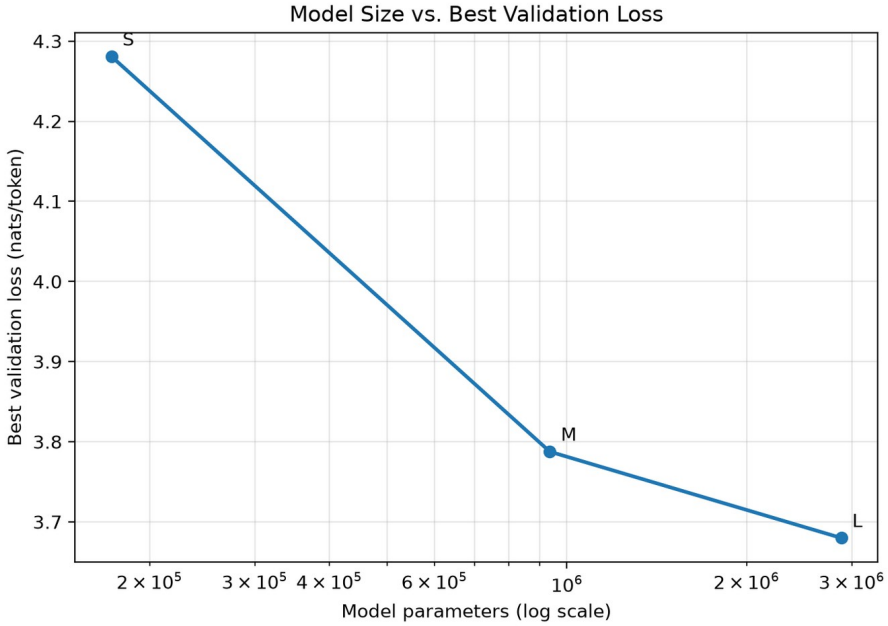
4.1 Training and Holdout-Shard Loss Curves



The loss curves illustrate how training progresses and how holdout-shard performance changes over time. Comparing the runs helps identify differences in convergence and generalization. The best holdout-shard losses are summarized in Table 1. The larger L model achieves the lowest recorded holdout-shard loss, while the higher learning rate in A3 results in a worse final best loss than the M baseline.

4.2 Model Scaling

The scaling comparison shows that holdout-shard loss decreases as model size increases from S to M to L. The largest model, L, has 2,880,384 parameters and achieves a best holdout-shard loss of 3.6799, compared with 4.2807 for S. The improvement from M to L is smaller than the improvement from S to M, suggesting diminishing gains over the tested model sizes.



4.3 KV-Cache Decoding Benchmark

Greedy decoding was tested with and without key-value (KV) caching. The cached and uncached implementations produced matching greedy outputs, supporting the correctness of the caching implementation for the tested case.

Decoding method	Throughput
KV cache enabled	639.59 tokens/sec
KV cache disabled	336.45 tokens/sec
Speedup	**1.90×**

KV caching improves decoding throughput by reusing keys and values computed for previous tokens instead of recomputing the full prefix at every generation step. The measured speedup demonstrates the practical performance benefit of this optimization on the tested CPU setup.

Appendix: Generated Samples

Model: Run L
Temperature: 0.8
Purpose: Inspect the quality and coherence of generated text.

Sample 1

```
ROMEO: but, if my say
Stand stay by the holy way and brain to his fast thee;
Seeming him in our honour that found by him,
Northumber-settle peace to thin him.
```

Sample 2

```
FRIAR LAURENCE:
Now plate is brow to this;
Who I fear thought to drawn her boy.
```

Sample 3

```
HATHASTINGS:
Sir Pruch God's grust
```

Observed failure pattern

The model imitates the dialogue-like structure of the training text, but produces malformed words and incoherent phrases, such as “Northumber-settle.” This suggests that the model has learned some surface patterns without consistently producing meaningful context. The limited model capacity and context length of 128 tokens may contribute to this behavior.